

**COLLEGIUM WITELONA
Uczelnia Państwowa**

**Wydział Nauk Technicznych i Ekonomicznych
Kierunek Informatyka
Specjalność Programowanie Aplikacji Mobilnych i Internetowych**

Sebastian Górski

**Nethelt – wieloplatformowy system monitorowania urządzeń sieciowych z modulem wykrywania anomalii
opartym na AI**

**Praca dyplomowa inżynierska
napisana pod kierunkiem
dr inż. Zbigniew Fryżlewicz**

Legnica 2026 rok

Spis treści

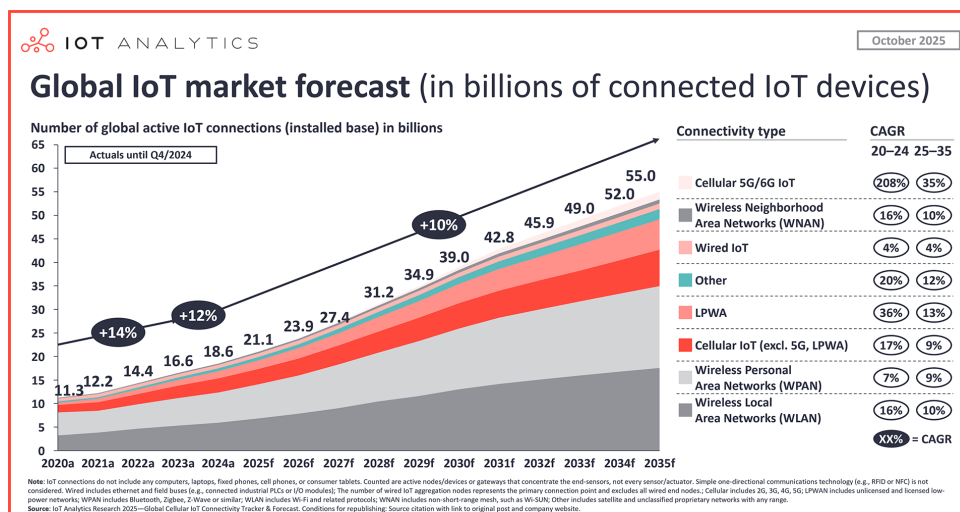
1	Wstęp	3
1.1	Wprowadzenie do problematyki	3
1.2	Cel pracy	4
1.3	Zakres pracy	4
1.4	Założenia projektowe	5
2	Założenia projektowe	7
2.1	Wprowadzenie	7
2.2	Przedmiot pracy i wizja aplikacji	7
2.3	Wymagania funkcjonalne	8
2.4	Wymagania niefunkcjonalne	9
2.5	Zarys architektury	10
2.5.1	Architektura logiczna	11
2.5.2	Architektura fizyczna	12

1. Wstęp

1.1 Wprowadzenie do problematyki

Aktualnie nie sposób wyobrazić sobie firmy, przedsiębiorstwa, a nawet gospodarstwa domowego, w którym nie korzysta się z internetu. Dostęp do zewnętrznych serwisów i usług stał się naszą codziennością. Korzystamy z internetu oraz urządzeń, łączących się do niego każdego dnia. W domach internet służy nam często do rozrywki i ułatwiania codziennych czynności, do wyszukiwania informacji i rozwiązywania problemów. Dzięki niemu jesteśmy w stałym kontakcie z innymi, mamy dostęp do płatności elektronicznych, a zakupy możemy zrobić nie wychodząc z domu. W pracy korzystamy z niego w celu wyszukiwania informacji i przesyłania ich dalej, wysyłania maili, pobierania danych z Państwowych spółek, czy chociażby robienia wideokonferencji z klientami z całego świata.

Poza dostępem do zewnętrznych usług coraz większą popularność zyskują urządzenia Smart oraz IoT (Internet of Things). Wielu z nas nawet nie ma pojęcia na temat tego jak wiele takich urządzeń znajduje się w naszym otoczeniu. Według IoT Analytics pod koniec 2025 r. liczba podłączonych urządzeń IoT na świecie miała osiągnąć 21,1 miliarda To wzrost o ok. 14% w stosunku do roku poprzedniego **TODO: zweryfikować dane przed oddaniem pracy**. Według prognozy liczba ta będzie tylko rosła i w 2030 roku może sięgnąć 39 miliardów[www1]. Tendencję tę można zaobserwować na rysunku 1.1 przedstawiający liczbę aktywnych urządzeń typu IoT wraz z prognozą na przyszłe lata.



Rysunek 1.1: Liczba urządzeń typu IoT w latach 2020–2035
 Źródło: <https://iot-analytics.com/number-connected-iot-devices/>

Pojawienie się tak dużej ilości urządzeń, z których specyfikacji wynika, że wręcz wymagają stałego połączenia z siecią, tworzy nowe problemy i pytania. Kluczowym pytaniem, które powinniśmy sobie zadać, jest to, czy przy utracie łączności z siecią takiego urządzenia pojawią się poważne komplikacje. Chociaż w przypadku np. lodówki bądź ekspresu do kawy działającym w sieci, brak połączenia nie wyrządzi sporych szkód, tak jednak w przypadku maszyn na hali produkcyjnej lub sygnalizacji świetlnej w centrum miasta, szkody mogą być spore i mogą zagrażać nie tylko sytuacji majątkowej, ale też bezpieczeństwu ludzi. W związku z powyższym należy monitorować stan takich urządzeń, a w przypadku ich awarii jak najszybciej dążyć do naprawy usterki.

1.2 Cel pracy

Celem niniejszej pracy jest stworzenie systemu wielomodułowego do monitorowania i zarządzania urządzeniami sieciowymi w środowisku lokalnym (LAN). System ma umożliwiać rejestrację i logowanie użytkowników, konfigurację połączeń sieciowych oraz monitorowanie stanu urządzeń.

Dodatkowym celem jest opracowanie modułu detekcji anomalii z wykorzystaniem algorytmu *Isolation Forest*, który pozwoli na wczesne wykrywanie nieprawidłowości w funkcjonowaniu sieci. System powinien wspierać dwa typy użytkowników – administratora oraz użytkownika zwykłego – z odpowiednimi uprawnieniami do zarządzania systemem i dostępem do danych.

Projekt obejmuje także wdrożenie aplikacji webowej w środowisku chmurowym, zapewniając dostępność dla użytkowników końcowych niezależnie od lokalizacji. Aplikacja desktopowa będzie dostępna do pobrania bezpośrednio na stronie internetowej.

1.3 Zakres pracy

Zakres pracy obejmuje:

- implementację aplikacji webowej w technologii Java 25 z użyciem frameworka Spring

Boot oraz interfejsu w TypeScript i Angular 21,

- stworzenie desktopowego klienta w Javie 25, działającego w trybie serwisu, umożliwiającego komunikację z serwerem,
- opracowanie modułu AI w Pythonie, wykorzystującego algorytm *isolation forest* do detekcji typowych anomalii w sieci,
- integrację modułów z centralną bazą danych PostgreSQL 18.2, współdzieloną między aplikacjami,
- wdrożenie aplikacji w środowisku chmurowym (Google Cloud Console) w celu zapewnienia dostępności dla użytkowników.

W zakresie pracy nie przewiduje się:

- stworzenia mobilnej aplikacji,
- obsługi wszystkich możliwych typów anomalii w sieci – moduł AI ograniczony jest do najczęściej występujących przypadków,
- rozbudowanego GUI w aplikacji desktopowej - wersja komputerowa będzie dostępna w trybie konsolowym. Wersja z interfejsem użytkownika zostanie zaimplementowana, jeśli pozostałe moduły będą gotowe przed ostatecznym terminem.

Zakres dokumentacji obejmuje przygotowanie instrukcji użytkownika, dokumentacji technicznej, diagramów UML oraz podstawowych statystyk działania systemu.

1.4 Założenia projektowe

Projekt ma charakter wielomodułowy i obejmuje trzy główne komponenty:

- aplikację webową,
- aplikację desktopową,
- moduł detekcji anomalii oparty na algorytmie AI typu *isolation forest*.

System zostanie zaprojektowany w taki sposób, aby umożliwić użytkownikowi rejestrację, logowanie, konfigurację połączenia ze swoją siecią LAN, dodawanie lokalnych urządzeń sieciowych, monitorowanie ich stanu oraz zmianę ustawień powiadomień. Aplikacja desktopowa będzie pełniła rolę klienta komunikującego się z serwerem, co jest niezbędne do prawidłowego funkcjonowania systemu w środowisku wielomodułowym. Dodatkowym wyzwaniem projektu będzie integracja wytrenowanego modelu AI, który będzie wspierał wczesne wykrywanie anomalii w sieci.

System przewiduje co najmniej dwa typy użytkowników: administratora oraz użytkownika zwykłego. Zalogowanie do systemu będzie obowiązkowe, a proces logowania zostanie udostępniony zarówno poprzez protokół OAuth2 (Google), jak i w formie lokalnej autoryzacji. Administrator będzie posiadał rozszerzone uprawnienia, w tym możliwość zarządzania użytkownikami oraz przeglądania podstawowych statystyk dotyczących korzystania z systemu.

Aplikacja webowa będzie realizowana w oparciu o nowoczesną wersję języka Java (wersja 25) oraz framework Spring Boot (w wersji 4.x.x), który ułatwia implementację i utrzymanie systemu. Interfejs użytkownika zostanie zbudowany przy użyciu TypeScript oraz frameworka Angular (wersja 21). Moduł AI zostanie opracowany w języku Python z wykorzystaniem odpowiednich bibliotek do uczenia maszynowego, co przyspieszy proces modelowania i analizy danych. Aplikacja desktopowa zostanie stworzona w tej samej wersji Javy, z której korzysta program internetowy, co pozwoli na jej uruchamianie na różnych systemach operacyjnych.

Dane pomiędzy modułami systemu będą współdzielone poprzez centralną bazę danych, której silnik wykorzysta PostgreSQL w wersji 18.2.

Projekt przewiduje wdrożenie aplikacji w środowisku chmurowym w celu zapewnienia dostępności dla użytkowników końcowych z dowolnej lokalizacji. Wstępnie planowane jest wykorzystanie platformy Google Cloud Console do tego celu.

2. Założenia projektowe

2.1 Wprowadzenie

W niniejszym rozdziale przedstawiono założenia projektowe systemu NetHelt, opracowanego w ramach pracy inżynierskiej. Rozdział obejmuje opis przedmiotu pracy, wizję tworzonej aplikacji, wymagania funkcjonalne i нефункционалне oraz zarys architektury logicznej i fizycznej systemu.

Celem przedstawionych założeń jest określenie sposobu realizacji systemu monitorowania urządzeń sieciowych działających w środowisku lokalnym. Opisane zostały zarówno wymagania dotyczące funkcjonalności aplikacji, jak również aspekty związane z bezpieczeństwem, wydajnością oraz komunikacją pomiędzy poszczególnymi modułami systemu.

Projektowany system ma charakter wielomodułowy i składa się z aplikacji webowej, aplikacji desktopowej działającej jako klient monitorujący oraz modułu analizy danych wykorzystującego algorytmy sztucznej inteligencji do wykrywania anomalii sieciowych. W rozdziale przedstawiono również ogólną koncepcję architektury rozwiązania oraz relacje pomiędzy jego komponentami.

Założenia opisane w dalszej części pracy stanowią podstawę do implementacji systemu oraz realizacji poszczególnych etapów projektu.

2.2 Przedmiot pracy i wizja aplikacji

Przedmiotem pracy jest opracowanie wielomodułowego systemu informatycznego do monitorowania urządzeń sieciowych działających w środowisku lokalnym (LAN). System ma umożliwiać kontrolę stanu urządzeń, analizę jakości połączenia sieciowego oraz wykrywanie anomalii mogących świadczyć o problemach w działaniu infrastruktury sieciowej.

Projektowane rozwiązanie składa się z aplikacji webowej, aplikacji desktopowej oraz modułu analizy danych wykorzystującego algorytm Isolation Forest. Aplikacja webowa odpowiada za zarządzanie systemem i prezentację danych użytkownikowi, natomiast aplikacja desktopowa realizuje operacje monitorujące w sieci lokalnej i przesyła wyniki do centralnego serwera.

System przewiduje obsługę użytkowników standardowych oraz administratorów, posiadających rozszerzone uprawnienia zarządzania systemem. Aplikacja ma umożliwiać m.in. konfigurację urządzeń, monitorowanie ich dostępności, analizę parametrów połączenia oraz wysyłanie powiadomień o wykrytych problemach.

Wizja projektu zakłada stworzenie skalowalnego i łatwego w rozbudowie systemu, dostępnego z poziomu przeglądarki internetowej oraz wdrożonego w środowisku chmurowym.

Komunikacja pomiędzy modułami będzie realizowana z wykorzystaniem interfejsów API oraz centralnej bazy danych PostgreSQL.

2.3 Wymagania funkcjonalne

Wymagania funkcjonalne określają funkcje oraz operacje, które system powinien realizować z perspektywy użytkownika. Ich celem jest opisanie zachowania aplikacji oraz możliwości udostępnianych poszczególnym aktorom systemu.

W procesie analizy wymagań wykorzystano koncepcję *user stories*, stosowaną w metodykach zwinnych do opisu oczekiwań użytkowników wobec systemu [www2]. Na podstawie przeprowadzonej analizy opracowano zestaw wymagań funkcjonalnych dla projektowanej aplikacji.

W systemie wyróżniono trzech głównych aktorów:

- administratora,
- użytkownika,
- gościa.

Administrator posiada uprawnienia do zarządzania użytkownikami oraz konfiguracją systemu. Użytkownik może zarządzać własnymi urządzeniami i konfiguracją monitorowania. Gość jest niezalogowanym użytkownikiem aplikacji i posiada dostęp wyłącznie do podstawowych funkcji systemu.

Zestaw wymagań funkcjonalnych przedstawiono w tabeli 2.1.

Tabela 2.1: Wymagania funkcjonalne systemu

ID	User Story
F-G1	Jako gość, chcę zobaczyć informację na temat aplikacji, aby zrozumieć jej przeznaczenie
F-G2	Jako gość, chcę się zarejestrować, aby uzyskać dostęp do systemu
F-G3	Jako gość, chcę się zalogować, aby korzystać z funkcji systemu
F-G4	Jako gość, chcę zresetować hasło, aby odzyskać dostęp do konta
F-U1	Jako użytkownik, chcę dodać urządzenie, aby je monitorować
F-U2	Jako użytkownik, chcę przeglądać listę swoich urządzeń, aby mieć wgląd w ich stan
F-U3	Jako użytkownik, chcę usunąć urządzenie, aby utrzymać porządek
F-U4	Jako użytkownik, chcę edytować dane urządzenia, aby zachować aktualność informacji
F-U5	Jako użytkownik, chcę konfigurować powiadomienia, aby otrzymywać informację na bieżąco
F-U6	Jako użytkownik, chcę przeglądać statystyki moich urządzeń, aby w łatwy sposób kontrolować czasy odpowiedzi
F-U7	Jako użytkownik, chcę pobrać klienta desktopowego, aby skonfigurować połączenie w mojej sieci
F-U8	Jako użytkownik, chcę zmienić swoje hasło, aby zachować bezpieczeństwo konta
F-U9	Jako użytkownik, chcę się wylogować, aby zablokować nieautoryzowany dostęp do mojego konta
F-A1	Jako administrator, chcę przeglądać listę użytkowników, aby mieć wgląd w konta w systemie
F-A2	Jako administrator, chcę usuwać i przywracać konta, aby zapewnić bezpieczeństwo systemu
F-A3	Jako administrator, chcę zmieniać role użytkowników, aby kontrolować ich uprawnienia
F-A4	Jako administrator, chcę resetować hasła użytkowników, aby umożliwić odzyskanie dostępu
F-A5	Jako administrator, chcę ręcznie wywoływać zadania z harmonogramu, aby mieć większą kontrolę nad przebiegiem działania aplikacji
F-A6	Jako administrator, chcę przeglądać ilość aktywnych użytkowników w danym okresie, aby być świadomym poziomu sukcesu aplikacji

Źródło: opracowanie własne.

2.4 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają ograniczenia oraz cechy jakościowe systemu informatycznego. W przeciwieństwie do wymagań funkcjonalnych nie opisują one konkretnych funkcji aplikacji, lecz definiują sposób działania systemu, jego wydajność, bezpieczeństwo, niezawodność oraz dostępność [www3].

Wymagania niefunkcjonalne mają istotny wpływ na jakość projektowanego rozwiązania oraz komfort korzystania z systemu przez użytkowników. Ich analiza została przeprowadzona na podstawie literatury z zakresu inżynierii oprogramowania oraz założeń projektowych aplikacji [ks1].

Najważniejsze wymagania niefunkcjonalne projektowanego systemu przedstawiono w tabeli 2.2.

Tabela 2.2: Wymagania niefunkcjonalne systemu

Kategoria	Wymaganie
Wydajność	System powinien obsługiwać minimum 100 równoczesnych użytkowników
Wydajność	Czas odpowiedzi API powinien być krótszy niż 500 ms dla 95% zapytań
Wydajność	System powinien wykorzystywać mechanizm cache'owania najczęściej używanych danych w celu redukcji czasu odpowiedzi o co najmniej 30%
Wydajność	Moduł AI powinien przetwarzać jednogodzinne okno danych pojedynczego urządzenia w czasie krótszym niż 100 ms
Dostępność	System powinien być dostępny przez minimum 99% czasu w skali miesiąca
Dostępność	Maksymalny jednorazowy czas niedostępności systemu nie powinien przekraczać 1 godziny
Bezpieczeństwo	System powinien wykorzystywać autoryzację użytkownika opartą o JWT oraz OAuth2
Bezpieczeństwo	Komunikacja pomiędzy klientem a serwerem powinna być szyfrowana z wykorzystaniem protokołu HTTPS
Bezpieczeństwo	Hasła użytkowników powinny być przechowywane w postaci skrótu kryptograficznego z wykorzystaniem algorytmu Argon2
Niezawodność	System powinien zapisywać logi błędów oraz umożliwiać ich przeglądanie administratorowi
Niezawodność	System powinien posiadać mechanizm ponawiania połączeń z urządzeniami obejmujący minimum 3 próby

Źródło: opracowanie własne.

2.5 Zarys architektury

Architektura systemu została zaprojektowana w sposób warstwowy, zgodnie z podejściem opisywanym przez R.C. Martina, które zakłada separację logiki biznesowej od szczegółów implementacyjnych oraz infrastruktury systemowej [ks2]. Takie podejście umożliwia utrzymanie wysokiej spójności systemu oraz ułatwia jego dalszy rozwój i testowanie.

2.5.1 Architektura logiczna

Architektura logiczna opisuje strukturę systemu z perspektywy jego komponentów funkcjonalnych oraz relacji między nimi, niezależnie od środowiska uruchomieniowego. Koncentruje się na podziale odpowiedzialności oraz przepływie danych pomiędzy modułami.

System został podzielony na następujące elementy logiczne:

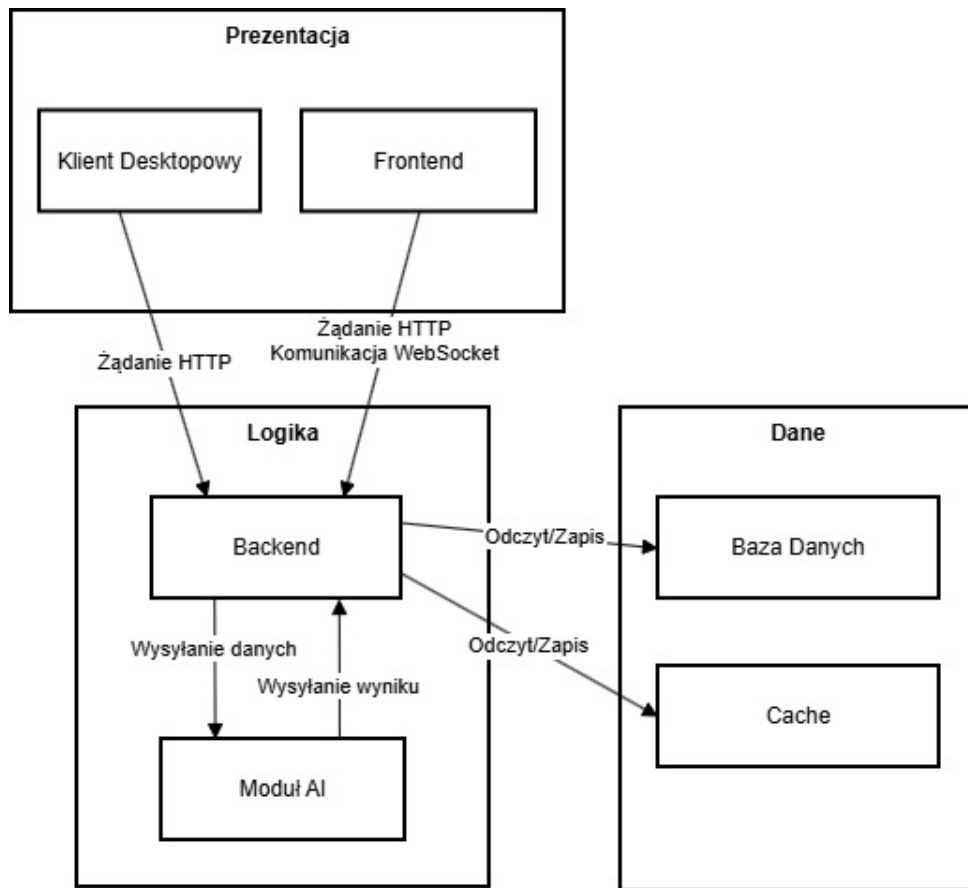
- Frontend (aplikacja webowa),
- Backend (REST API + WebSocket),
- Moduł AI,
- Baza danych,
- Cache,
- Klient desktopowy.

Elementy te zostały zorganizowane w trzy warstwy:

- warstwa prezentacji,
- warstwa logiki aplikacyjnej,
- warstwa danych.

Warstwa prezentacji obejmuje frontend oraz klienta desktopowego, które odpowiadają za interakcję z użytkownikiem. Warstwa logiki obejmuje backend oraz moduł AI, które realizują przetwarzanie żądań, logikę biznesową oraz analizę danych. Warstwa danych obejmuje bazę danych oraz mechanizm cache, odpowiedzialne za przechowywanie danych w sposób trwały oraz tymczasowy.

Komunikacja pomiędzy komponentami przebiega w następujący sposób: frontend komunikuje się z backendem poprzez REST API oraz WebSocket, zapewniając komunikację w czasie rzeczywistym, natomiast klient desktopowy wykorzystuje REST API. Backend przetwarza żądania i deleguje zadania do modułu AI w celu analizy danych, a następnie zapisuje lub odczytuje dane z warstwy danych.



Rysunek 2.1: Architektura logiczna systemu
Źródło: opracowanie własne

2.5.2 Architektura fizyczna

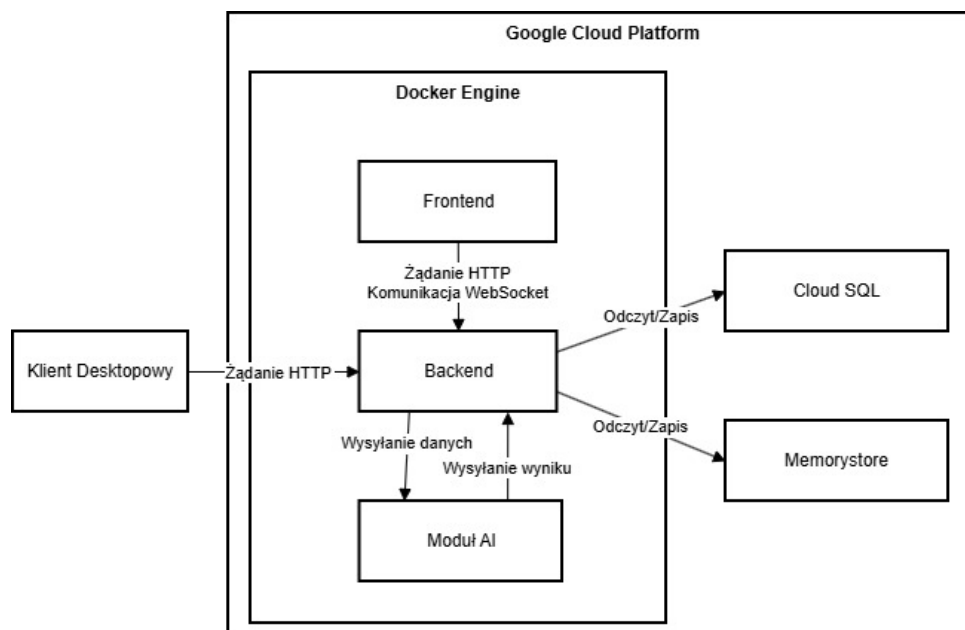
Architektura fizyczna przedstawia sposób wdrożenia systemu w środowisku infrastrukturalnym. System został zaimplementowany z wykorzystaniem platformy Google Cloud Platform (GCP), która zapewnia skalowalne środowisko uruchomieniowe oraz usługi zarządzane.

Poszczególne komponenty systemu zostały skonteneryzowane przy użyciu technologii Docker. Każdy główny moduł działa w osobnym kontenerze, co zapewnia ich izolację oraz możliwość niezależnego wdrażania.

W warstwie fizycznej system obejmuje:

- backend uruchamiany w kontenerze Docker,
- frontend uruchamiany w kontenerze Docker,
- moduł AI uruchamiany w kontenerze Docker,
- bazę danych realizowaną w usłudze Cloud SQL,
- cache realizowany w usłudze Memorystore,
- klienta desktopowego instalowanego lokalnie na urządzeniu użytkownika.

Takie podejście zapewnia separację komponentów, łatwość skalowania oraz zwiększoną odporność systemu na awarie, a także upraszcza proces wdrażania i utrzymania infrastruktury.



Rysunek 2.2: Architektura fizyczna systemu
Źródło: opracowanie własne

Spis rysunków

1.1	Liczba urządzeń typu IoT w latach 2020–2035 Źródło: https://iot-analytics.com/number-connected-iot-devices/	4
2.1	Architektura logiczna systemu Źródło: opracowanie własne	12
2.2	Architektura fizyczna systemu Źródło: opracowanie własne	13

Spis tabel

2.1	Wymagania funkcjonalne systemu	9
2.2	Wymagania niefunkcjonalne systemu	10

Spis listingów

Bibliografia

Wydawnictwa zwarte

- [ks1] I. Sommerville, *Inżynieria oprogramowania*, wyd. 10, Wydawnictwo Naukowe PWN, Warszawa, 2016.
- [ks2] R.C. Martin, *Czysta architektura. Struktura i design oprogramowania. Przewodnik dla profesjonalistów*, Wydawnictwo Helion, Gliwice, 2018.

Netografia

- [www1] [www1] S. Sinha, *State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally*, IoT Analytics, 2025, <https://iot-analytics.com/number-connected-iot-devices/> [dostęp: 07.12.2025].
- [www2] [www2] Atlassian, *User stories*, <https://www.atlassian.com/pl/agile/project-management/user-stories> [dostęp: 25.05.2026].
- [www3] Perforce, *What Are Non-Functional Requirements?*, <https://www.perforce.com/blog/alm/what-are-non-functional-requirements-examples> [dostęp: 25.05.2026].